# MICROSAR Classic vLinkGen

## Technical Reference

### Compiler Specific Linker Script Generator

Version 2.11.0

| | |
|---|---|
| Authors | virbse, visjhr |
| Status | Released |

# Document Information

## History

| Author | Date | Version | Remarks |
|---|---|---|---|
| visces | 2018-10-16 | 1.0.0 | First Release. |
| visces | 2018-11-22 | 1.1.0 | Support Memory Region Blocks.<br>Extended RAM zero initialization. |
| virbse | 2019-01-14 | 1.2.0 | Added new supported compilers.<br>Updated to latest template version.<br>Updated chapter *Introduction.* |
| virbse | 2019-02-28 | 1.3.0 | Added new supported compilers.<br>Added chapter *Group Pre- & Post-User Code.*<br>Added chapter *Memory Partitioning.*<br>Added chapter *Compiler Restrictions.*<br>Added chapter *Output Generation Order.*<br>Added chapter *Label Generation.* |
| virbse | 2019-05-03 | 1.4.0 | Added new supported compilers.<br>Added new supported Features.<br>Added chapter *Initialization Tables.*<br>Added chapter *Type Definitions.*<br>Updated chapter *Scope of Delivery.*<br>Updated chapter *Group Preprocessor Conditions.* |
| virbse, visdqk | 2019-07-05 | 1.5.0 | Added new supported compilers.<br>Added chapter *Section Group Gaps.* |
| virbse | 2019-07-08 | 1.6.0 | Added new supported compilers.<br>Updated chapter *Compiler Restrictions.* |
| virbse | 2020-06-10 | 2.0.0 | Updated to new vLinkGen generation.<br>Updated to latest template version. |
| virbse | 2020-06-16 | 2.0.1 | Updated chapter *Migration of Existing Projects.* |
| virbse | 2020-08-26 | 2.1.0 | Added new supported Features.<br>Added chapter *Special Section Generation*.<br>Added chapter *Workaround for TCVX-39968*.<br>Updated chapter *Compiler Restrictions*.<br>Updated *Available Init Stages* table. |
| virbse | 2020-11-02 | 2.2.0 | Added new supported compilers.<br>Minor editorial changes. |
| virleh | 2021-01-11 | 2.2.1 | Updated chapter *Type Definitions.* |
| virbse | 2021-03-18 | 2.3.0 | Added new supported compilers.<br>Removed chapter *Architecture Overview*. |
| virbse | 2021-07-07 | 2.3.1 | Updated chapter *Compiler Restrictions*. |

| | | | |
|---|---|---|---|
| virbse | 2021-10-07 | 2.4.0 | Added new supported compilers.<br>Updated chapter *Supported Compilers*.<br>Added chapter *Metadata Based Configuration*.<br>Added chapter *Reordered Groups after Linking*. |
| virbse | 2022-01-03 | 2.5.0 | Added new supported compilers.<br>Renamed chapter *Compiler Restrictions* to *Compiler Specialities* and updated it. |
| virleh | 2022-09-20 | 2.5.1 | Name change: “MICROSAR” to “MICROSAR Classic”. |
| virleh | 2022-11-29 | 2.6.0 | Added more information about the ARM5/6 restrictions and generation order |
| virleh | 2022-02-02 | 2.6.1 | Added disclaimer to state that vLinkGen only generates template (example) code |
| visjhr | 2023-02-23 | 2.6.2 | Replaced content of chapter 5 “Configuration“ with a reference to the vLinkGen User Manual. |
| visjhr | 2023-09-15 | 2.7.0 | Added new supported compiler.<br>Updated chapter *ARM*. |
| visjhr | 2024-08-14 | 2.8.0 | Added init stage POWER_ON |
| visjhr | 2024-11-20 | 2.9.0 | Added file generation method MODULE_SNIPPETS_GROUPWISE |
| visjhr | 2025-01-31 | 2.10.0 | Added file generation method STANDARD_WITH_VARIANTES, changed file generation method STANDARD |
| visjhr | 2025-05-13 | 2.11.0 | Added  KEEP flag to GCC SpecSections |

## Reference Documents

| No. | Source | Title | Version |
|---|---|---|---|
| [1] | AUTOSAR | General Specification of Basic Software Modules | R4.2.1 |
| [2] | Vector | MICROSAR Classic vBaseEnv User Manual | See delivery information |



**Caution**
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

## Contents

**1 Introduction ..... 7**
1.1 Supported Compilers ..... 8
1.2 Definitions of Terms ..... 8

**2 Functional Description ..... 11**
2.1 Features ..... 11
2.1.1 File Generation ..... 11
2.1.1.1 Notes on Snippets ..... 12
2.1.2 Variant Handling ..... 12
2.1.2.1 Variant Specific Mappings ..... 13
2.1.3 User Code ..... 15
2.1.4 Initialization Tables ..... 16
2.1.4.1 ECC Memory Initialization ..... 17
2.1.4.2 Variable Initialization ..... 17
2.1.4.3 RAM Code Execution ..... 17
2.1.5 Metadata Based Configuration ..... 18
2.2 Initialization ..... 19
2.3 States ..... 19
2.4 Main Functions ..... 19
2.5 Error Handling ..... 19
2.5.1 Development Error Reporting ..... 19
2.5.2 Production Code Error Reporting ..... 19

**3 Integration ..... 20**
3.1 Embedded Implementation ..... 20
3.2 Compiler Specialities ..... 21
3.2.1 ARM ..... 21
3.2.1.1 Reordered Groups ..... 22
3.2.2 Green Hills ..... 22
3.2.3 Renesas ..... 22
3.2.4 Tasking ..... 23
3.2.4.1 Special Flags ..... 23
3.2.4.2 Reordered Groups after Linking ..... 23
3.2.4.2.1 Combine Logical Groups ..... 24
3.2.4.2.2 Split Memory Regions ..... 24
3.2.4.3 Workaround for TCVX-39968 ..... 24
3.2.4.3.1 Error Description ..... 24
3.2.4.3.2 Preconditions ..... 25
3.2.4.3.3 Workaround ..... 25

3.2.5 Texas Instruments ..... 26
3.3 Output Generation Order ..... 26
3.4 Label Generation ..... 27
3.5 Special Section Generation ..... 28
3.6 Migration of Existing Projects ..... 28

**4 API Description ..... 31**
4.1 Type Definitions ..... 31
4.1.1 vLinkGen_MemArea ..... 31
4.1.2 vLinkGen_MemAreaSet ..... 31
4.1.3 vLinkGen_RamMemArea ..... 31
4.1.4 vLinkGen_RamMemAreaSet ..... 32

**5 Configuration ..... 33**

**6 Glossary and Abbreviations ..... 34**
6.1 Glossary ..... 34
6.2 Abbreviations ..... 34

**7 Contact ..... 35**

## Illustrations

Figure 1-1 Schematic Configuration Example ..... 10

## Tables

Table 1-1 Supported Compilers ..... 8
Table 2-1 File Generation Types ..... 12
Table 2-2 User Group Placement Options ..... 15
Table 2-3 Initialization Tables ..... 16
Table 2-4 Available Init Stages ..... 17
Table 3-1 Implementation files ..... 20
Table 3-2 Generated linker scripts and snippets ..... 20
Table 3-3 Tasking Special Flags ..... 23
Table 3-4 Output generation order ..... 26
Table 3-5 Label Generation ..... 27
Table 3-6 Special Section Generation ..... 28
Table 4-1 vLinkGen_MemArea ..... 31
Table 4-2 vLinkGen_MemAreaSet ..... 31
Table 4-3 vLinkGen_RamMemArea ..... 32
Table 4-4 vLinkGen_RamMemAreaSet ..... 32
Table 6-1 Glossary ..... 34
Table 6-2 Abbreviations ..... 34

# 1 Introduction

This document describes the functionality, API and configuration of the MICROSAR BSW module vLinkGen.

| | | |
|---|---|---|
| **Supported Configuration Variants:** | pre-compile | |
| **Vendor ID:** | vLinkGen_VENDOR_ID | 30 decimal<br>(= Vector-Informatik, according to HIS) |
| **Module ID:** | vLinkGen_MODULE_ID | 255 decimal<br>(according to ref. [1]) |

The vLinkGen (Vector linker script generator) module shall provide an abstract way to create linker scripts, so that code, constants and variables (linker symbols) in a binary are placed at the intended location in the target memory (e.g. ROM, RAM, etc.).

The basic idea of vLinkGen is to have a compiler independent description (except for assignment to hardware specific memory areas) of the linker script to be able to apply it in a similar way for several projects.

The configuration of the linker script can be done completely without knowledge of the compiler specific linker script language. The vLinkGen then generates the linker script in the appropriate format for the selected compiler.



**Caution**
The linker scripts and code generated by vLinkGen are example code. This example code is only intended for illustrating an example of a possible BSW integration and BSW configuration. The example code has not passed any quality control measures and may be incomplete. The example code is neither intended nor qualified for use in series production. The example code as well as any of its modifications and/or implementations must be tested with diligent care and must comply with all quality requirements which are necessary according to the state of the art before their use.

## 1.1 Supported Compilers

All compilers listed in the following table are supported. Entries in the **Version Restrictions** column other than "-" only indicate whether a compiler is supported only from or in a particular version.

| Compiler Name | Version Restrictions |
|---|---|
| ARM | only 5.x.x, 6.x.x or later |
| Wind River Diab | - |
| GCC | - |
| Green Hills | - |
| HighTec (GCC) | - |
| HighTec (LLVM) | - |
| IAR | only 7.x.x or later |
| Linaro | - |
| Microchip MPLAB XC32 | - |
| NXP (GCC) | - |
| Renesas | - |
| Tasking | - |
| Texas Instruments | - |
| Texas Instruments (LLVM) | - |
| Ccac | Following version or later:<br>DesignWare ARC C/C++ Compiler T-2022.09<br>Driver Version: 2.04<br>Compiler Version: 14.0.6<br>Revision: (build 003) (LLVM 14.0.6)<br>Based on LLVM version: 14.0.6 |

Table 1-1 Supported Compilers



**Note**
Not all versions were tested for each compiler. It is assumed that all common versions will work with the generated linker scripts.

## 1.2 Definitions of Terms

This chapter describes the different terms in the vLinkGen and shows a schematic configuration example in Figure 1-1.

- **Hardware Memory Areas** (`vBaseEnvMemLayoutHwRegion`) are provided by vBaseEnv module. They represent the actual memory layout on the hardware and cannot be changed.

- **Memory Regions** (`vLinkGenMemoryRegion`), which can be assigned to the hardware memory areas, are a logical abstraction layer to separate the configuration from the actual memory layout and to make it hardware independent. See `Code_Flash` and `Data_Flash`, mapped to `FLASH` in Figure 1-1.
- Memory Regions can be divided into **Memory Region Blocks** (`vLinkGenMemoryRegionBlock`), which can be linked to the lower or upper boundary of the region. They are used for the actual memory definitions in the generated linker script. See `Code_Flash_0` and `Code_Flash_1`, mapped to `CodeFlash` in Figure 1-1.
- **Logical Groups** (`vLinkGenLogical*Group`) are a logical abstraction layer to group multiple Section Groups or Linker Symbols and map them to a Memory Region Block. See `Appl_Const`, mapped to `Data_Flash_0` in Figure 1-1.
- **Section Groups** (`vLinkGen*SectionGroup`) are used to group multiple Linker Sections of a kind together. In compiler manuals they are often called ‘Output Sections’. See `Code_Default` containing Linker Sections and is mapped to `Code_Flash_0` in Figure 1-1.

  The size of these groups depends on the concrete content of the Linker Sections. Also, their start address depends on the configured position and the size of previous groups, which are mapped to the same Memory Region Block.
- **Linker Sections** are the smallest entity in the vLinkGen and contain the concrete content of the different linker symbols (functions, constants, variables, etc.) assigned to them in the source code. In compiler manuals they are often called ‘Input Sections'.



**Caution**
The following figure shows only a schematic example of a configuration and is by no means to be regarded as a template or complete. It serves only as an illustration of the existing types and their relationships to each other.

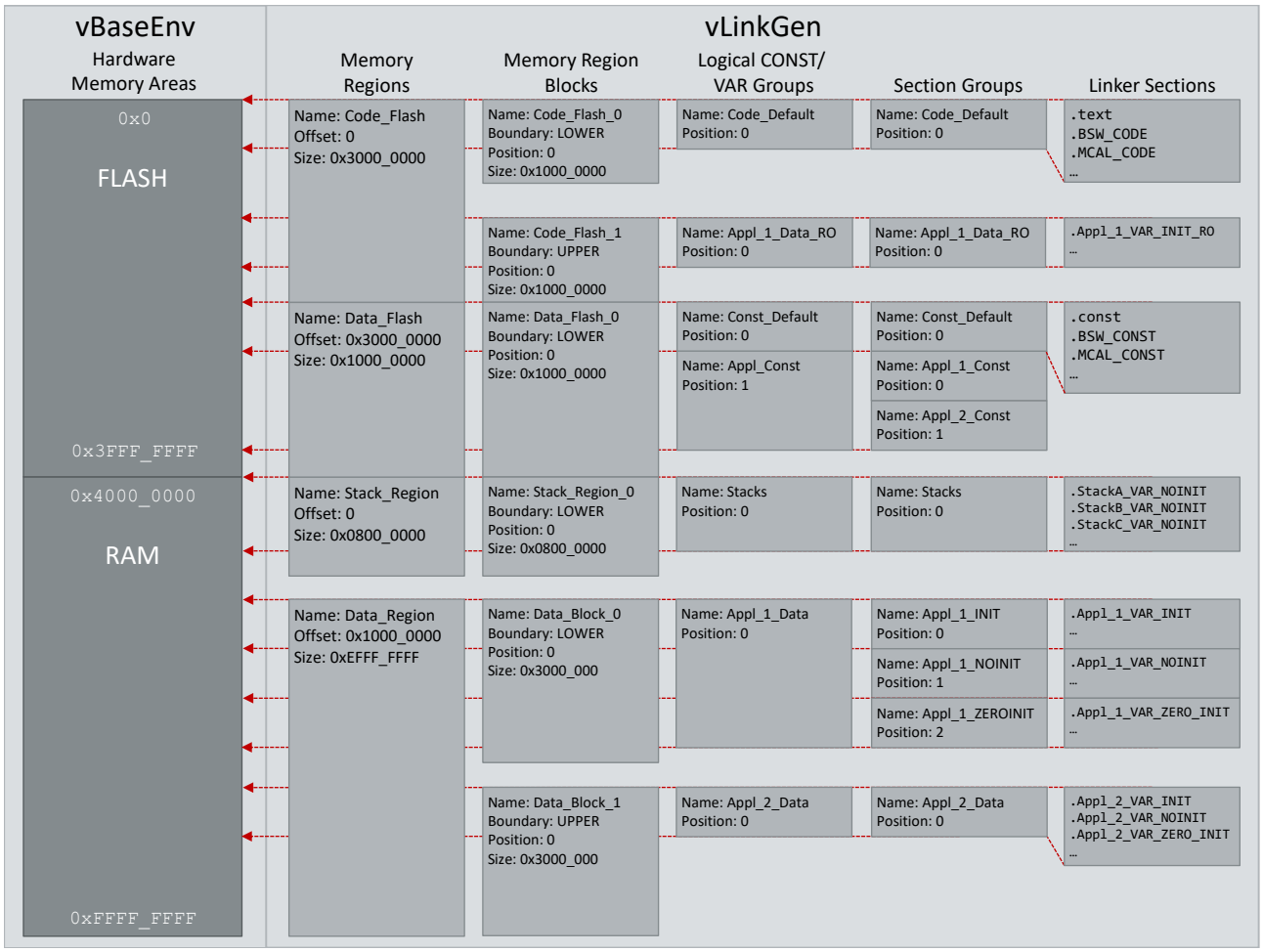



Figure 1-1 Schematic Configuration Example

# 2 Functional Description

The following chapters contain some detail descriptions of the vLinkGen features.



**Caution**
Please consider that not all features are supported by all compilers. If a feature is not supported, the corresponding parameters are either set to read-only or an error or warning message occurs when they are activated in the DaVinci Configurator.

## 2.1 Features

### 2.1.1 File Generation

The vLinkGen provides the possibility to generate the linker script in different variants with different content. These can be selected via the **File Generation** (`vLinkGen/vLinkGenGeneral/vLinkGenFileGeneration`) parameter in the configuration.

For details about the different file types and their content, please see Table 2-1 below.

The names with which the files are generated can be found in chapter 3.1.

Details about variant handling in vLinkGen can be found in chapter 2.1.2.

| | |
|---|---|
| **STANDARD** | Generates a complete compiler specific linker script.<br>▶ Eventually configured variant parameters of the logical groups are ignored. |
| **STANDARD_WITH_VARIANTS** | Generates a complete compiler specific linker script.<br>▶ Logical Groups that are assigned to one or more variants are encapsulated with C preprocessor macros. If variants are configured, this file may need to be preprocessed before it can be passed to the linker. This applies to all linkers that do not have their own preprocessor that can resolve C preprocessor macros (e.g. `#if defined (VariantName)`). Also the variant must be known during preprocessing. This means it must be passed to the preprocessor (e.g. via options with `-DVariantName`).<br>▶ Logical Groups that are not assigned to a variant are generated without macros and are therefore always active. |
| **ONE_FILE_PER_VARIANT** | Generates a complete compiler specific linker script per variant.<br>▶ These files can be used as they are and passed to the linker via specific option. Preprocessing of these files is not necessary.<br>▶ All Logical Groups that are assigned to one or more variants are generated in the corresponding variant specific files.<br>▶ Logical Groups that are not assigned to a variant are generated in all files and are therefore always active. |

| | |
|---|---|
| MODULE_SNIPPETS | Generates several linker script snippets containing all Logical Groups (including all Section Groups) with a module reference.<br>▶ Eventually configured variant parameters of the logical groups are ignored.<br>▶ Please consider the notes on snippets in chapter 2.1.1.1. |
| MODULE_SNIPPETS_GROUPWISE | Generates several linker script snippets containing all Logical Groups (including all Section Groups) with a module reference embraced with C preprocessor ifdefs based on the name of the corresponding Logical Group / Section Group.<br>▶ Eventually configured variant parameters of the logical groups are ignored.<br>▶ Please consider the notes on snippets in chapter 2.1.1.1. |
| VARIANT_SNIPPETS | Generates several linker script snippets containing all Logical Groups (including all Section Groups) of a variant.<br>▶ Please consider the notes on snippets in chapter 2.1.1.1. |
| LOGICAL_GROUP_SNIPPETS | Generate several linker script snippets containing all Section Groups of a Logical Group.<br>▶ Eventually configured variant parameters of the logical groups are ignored.<br>▶ Please consider the notes on snippets in chapter 2.1.1.1. |

Table 2-1 File Generation Types

#### 2.1.1.1 Notes on Snippets

- Snippets are not supported for all compilers.
- Snippets only contain the corresponding Logical Groups and nothing else.
- Snippets cannot be passed directly to the linker. They can only be included by other linker scripts.
  - However, the names of the Memory Region Blocks (e.g. FLASH) from the vLinkGen configuration must match the memory definitions from the linker script into which the snippets are to be included.
- Snippets never contain memory definitions and global (PRE or POST) user groups (See 2.1.3).

### 2.1.2 Variant Handling

Sometimes it can be necessary to compile a project in several build processes with different memory layouts. Therefore, vLinkGen provides the possibility to configure several variants of a linker script in a single project and then generate them separately. This means that several linker scripts with different memory layouts can be generated from one configuration. These files can then be used for the several build processes.

**Caution**
If variants are configured, exactly one of them must always be defined in a build process. For example, by passing it as compiler and/or linker option (e.g. `-DVariantName`).

Variants are configured globally, and then referenced by one or more Logical Groups (`vLinkGenLogical*Group`). The following points must be observed:

- A Logical Group can also be assigned to several variants. If it is, it is therefore valid in each of them.
- Logical Groups that are not assigned to a variant are to be understood as global. This means they are active in every variant.

Depending on the configured variants and the settings of the **File Generation** (see chapter 2.1), the corresponding linker scripts are then generated.

An example how variants are configured can be found in chapter 5.

**Example**
The following example shows a Logical Group (including one Section Group) which is assigned to two variants (Variant_A and Variant_B) and the **File Generation** is set to `STANDARD`.

```
#if defined (Variant_A) || defined (Variant_B)
  .MySectionGroup :
  {
    _MySectionGroup_START = ABSOLUTE(.);
    *(*.My_Section)
    _MySectionGroup_END = ABSOLUTE(. - 1);
    _MySectionGroup_LIMIT = ABSOLUTE(.);
  } > RAM

  _MyLogicalGroup_ALL_START = _MySectionGroup_START;
  _MyLogicalGroup_ALL_END = _MySectionGroup_END;
  _MyLogicalGroup_ALL_LIMIT = _MySectionGroup_LIMIT;
#endif
```

### 2.1.2.1 Variant Specific Mappings

In addition to the normal variant handling, there is also the possibility to link certain Logical Groups (respectively all their contained Section Groups) to different Hardware Memory Areas in different variants.

What means a Logical Group, for example, can be linked to memory area `RAM_A` in Variant A and to totally different memory area `RAM_B` in Variant B. So, there is no need to duplicate the Logical Group and map it to different Memory Region Blocks. See example below.

The following behavior applies when setting the Memory Region Block references during generation:

- For all variants, the "Default Memory Region Block" references from the Logical Group are used by default
- If a variant is configured in a specific Variant mapping (`vLinkGen*GroupVariantSpecifics/vLinkGen*GroupSpecificVariant`), then the "Overwriting Memory Region Block" references are used
- Where the "overwriting" references in the variant specifics are optional
  - This means that for a variant, for example, only the ROM/RAM copy can be linked elsewhere



**Note**
Variant Specific Mappings also work with all **File Generation** types specified in chapter 2.1.



**Example**
The following example shows a Logical Group (including one Section Group) which is assigned to two variants (Variant_A and Variant_B) and the **File Generation** is set to `STANDARD`. In addition the default Memory Region Block (`RAM_A`) is specifically overwritten to `RAM_B` for Variant_B. So `MySectionGroup` will be linked to different memory areas in the variants.

```
#if defined (Variant_A) || defined (Variant_B)
  .MySectionGroup :
  {
    _MySectionGroup_START = ABSOLUTE(.);
    *(*.My_Section)
    _MySectionGroup_END = ABSOLUTE(. - 1);
    _MySectionGroup_LIMIT = ABSOLUTE(.);
#if defined (Variant_A)
  } > RAM_A
#endif
#if defined (Variant_B)
  } > RAM_B
#endif

  _MyLogicalGroup_ALL_START = _MySectionGroup_START;
  _MyLogicalGroup_ALL_END = _MySectionGroup_END;
  _MyLogicalGroup_ALL_LIMIT = _MySectionGroup_LIMIT;
#endif
```

### 2.1.3 User Code

The vLinkGen offers the possibility to generate compiler specific user code (user code group) at almost any position in the linker script. This allows the generated linker script to be extended with additional code snippets. This could be for example include of another linker script or generate additional comments.

The **Placement** (`vLinkGenUserGroupPlacement`) provides different options to choose from to generate the user code group to a specific location in the linker script. For details about the available options, see Table 2-2.

| | |
|---|---|
| **PRE** | The user code is generated at the beginning of the linker script. |
| **GROUP_LIST** | The user code is generated within the Logical Groups list at the specified position. Note that this option is not available for all compilers. |
| **POST** | The user code is generated at the end of the linker script. |

Table 2-2 User Group Placement Options

To configure such user code groups, create a `vLinkGenLogicalUserGroup` and select where it should be generated via the **Placement**. The `vLinkGenUserSectionGroup`

contains the compiler specific code which is generated in the linker script exactly as configured.

### 2.1.4 Initialization Tables

Depending on the configuration, vLinkGen generates some structures (initialization tables) which can be used to:

- Initialize Memory Region Blocks with zeros (ECC initialization). See chapter 2.1.4.1.
- Initialize variables concrete values (INIT) or zeros (ZERO_INIT). See chapter 2.1.4.2.
- Copy code from ROM to RAM to be executed there. See chapter 2.1.4.3.

The initialization itself is usually done by the startup code of vBRS. This code takes over the writing of the memory areas with zeros or the copying of the values from ROM to RAM.

The following Table 2-3 shows the generated structures and their content.

The type definitions can be found in chapter 4.1.

| Name | Content |
|---|---|
| `vLinkGen_MemAreaSet` **`vLinkGen_ZeroInit_<Stage>_BlocksSet`** | Memory Region Blocks to be initialized with zeros at a certain startup stage. Available stages can be found in Table 2-4. |
| `vLinkGen_MemAreaSet` **`vLinkGen_ZeroInit_<Stage>_GroupsSet`** | VAR Section Groups to be initialized with zeros at a certain startup stage. Available stages can be found in Table 2-4. |
| `vLinkGen_RamMemAreaSet` **`vLinkGen_Init_<Stage>_GroupsSet`** | Section Groups to be copied from ROM to RAM at a certain startup stage. This can be VAR Section Groups with initialization data or CONST Section Groups with code to be executed from RAM. Available stages can be found in Table 2-4. |

Table 2-3 Initialization Tables

The following Table 2-4 shows all available initialization stages. The initialization stage determines the time of initialization during ECU startup.

It can be configured for each Memory Region Block, VAR or CONST Section Group separately via the **Init Stage** parameter. For Section Groups the value is only evaluated if the **Init Policy** is set to value other than `NONE`.

Consider that not all stages are available for Memory Region Blocks and CONST Section Groups.

| Init Stage | Description |
|---|---|
| `EARLY` | Initialize always directly after reset. Shall only be used if really necessary. Since the PLL might not be initialized at that time, the initialization of large areas may take a long time. |
| `ZERO` | Always initialized first during the default vBRS startup routine. |
| `HARD_RESET_ONLY` | Same initialization stage as ONE. However, initialization is only done if a hardware reset occurred. |

| | |
|---|---|
| ONE | Always initialized second during the default vBRS startup routine.By default, this value should be used if an initialization is required. |
| TWO, THREE | Optional init stages, which are initialized accordingly after the HARD_RESET_ONLY stage. Shall only be used if really necessary. One use case might be to reset the Wdg between the initialization of two stages. |
| POWER_ON | Initialized on cold start / PowerOn reset only. No initialization on warm start (e.g. software reset). |

Table 2-4 Available Init Stages

#### 2.1.4.1 ECC Memory Initialization

On some platforms it is necessary to initialize certain memory areas (ECC protected memory areas) before the first read or write access. This is usually done by writing the complete area with zeros. The vLinkGen provides the possibility to enable this initialization for each Memory Region Block.

An example how to configure zero initialized Memory Region Blocks can be found in chapter 5.

#### 2.1.4.2 Variable Initialization

The vLinkGen also offers the possibility to initialize variables on ECU startup with either predefined values or zeros. No initialization is of course also possible. However, the following points must be observed:

- Since the type of initialization is configured for a Section Group, it applies to all underlying Linker Sections. Which also means that within a Section Group there shall only be Linker Sections which need the same type of initialization.
- The type of initialization of a Section Group can only be INIT, ZERO_INIT or NONE at the same time. Multiple selection is not possible.

An example how to configure VAR Section Group to be initialized with predefined values or zero can be found in chapter 5.



**Note**
Some linkers provide the possibility to create own so-called "copy tables". These cannot be used together with vLinkGen and may have to be disabled via options.

#### 2.1.4.3 RAM Code Execution

Like the VAR Section Groups, vLinkGen also offers the possibility to copy CONST Section Groups from ROM to RAM. This may be necessary if code is to be executed in RAM.

An example how to configure a CONST Section Group to be copied from ROM to RAM can be found in chapter 5.

### 2.1.5 Metadata Based Configuration

This feature is intended to assist in setting up a correct vLinkGen configuration. Basically, it automatically assigns Logical Groups (CONST or VAR) to the correct Memory Region Blocks, respectively the correct Hardware Memory Areas.

It should help especially if new Logical Groups are created at configuration time (e.g., core-specific groups of the OS) or the tested derivative is changed in the vBaseEnv (`vBaseEnvTestedDerivativeRef`). The vLinkGen then uses the metadata to select the correct Memory Region Block for the Logical Group. The metadata of a Logical Group is described by the following parameters:

- Content type:
  `/vLinkGen/vLinkGenMemLayout/vLinkGenLogical*Group/vLinkGen*GroupContentType`
- Core Id:
  `/vLinkGen/vLinkGenMemLayout/vLinkGenLogical*Group/vLinkGen*GroupCore`

Whereas the latter parameter is only relevant for Logical Groups with core local content.

The vLinkGen comes with a set of predefined content types (e.g., `GLOBAL_CODE`, `LOCAL_DATA`, etc.) which can be used for the Logical Groups (`vLinkGen*GroupContentType`). This determines what type of content the Logical Group will contain after linking. For automatically created Logical Groups, the content type is set automatically.

In addition, the content types are also assigned to the Memory Region Blocks to indicate what kind of content can be linked into a Memory Region Block. Assigning the content types to the Memory Region Blocks is usually prepared by a recommended configuration and does not have to be done manually. The list of content types for a Memory Region Block is specified via the reference parameter:

`/vLinkGen/vLinkGenMemLayout/vLinkGenMemoryRegion/vLinkGenMemoryRegionBlock/vLinkGenMemoryRegionBlockContentType`

Via the set content type of a Logical Group, vLinkGen can select the correct reference for the Memory Region Block (`vLinkGen*GroupRegion`) and, if necessary, also for the copy Memory Region Block (`vLinkGenConstGroupRamRegion` or `vLinkGenVarGroupRomRegion`). Furthermore, the availability flag of the Memory Region is taken into account during the selection process. It indicates whether the referenced Hardware Memory Area of the Memory Region is available on the currently tested derivative or not. The automatically set availability flag is defined by the following parameter:

`/vLinkGen/vLinkGenMemLayout/vLinkGenMemoryRegion/vLinkGenMemoryRegionAvailable`



**Note**
The vLinkGen comes with a set of predefined content types that are already mapped correctly via a recommended configuration. However, it is always possible to define own content types and use them accordingly.



**Note**
As long as the content type of a Logical Group is set, the Memory Region Block is also set automatically. What means it is not changeable. Should it be necessary to change the referenced Memory Region Block, the content type can simply be deleted.

## 2.2 Initialization

The vLinkGen does not provide any C initialization functions.

## 2.3 States

The vLinkGen does not provide any states.

## 2.4 Main Functions

The vLinkGen does not provide any main functions.

## 2.5 Error Handling

### 2.5.1 Development Error Reporting

The vLinkGen does not provide development error reporting.

### 2.5.2 Production Code Error Reporting

The vLinkGen does not provide production code error reporting.

# 3 Integration

This chapter gives necessary information for the integration of the MICROSAR vLinkGen into an application environment of an ECU.

## 3.1 Embedded Implementation

The delivery of the vLinkGen contains these source code files:

| File Name | Description | Integration Tasks |
|---|---|---|
| vLinkGen_Lcfg.c | Generated file that contains initialization tables (see 2.1.4). | - |
| vLinkGen_Lcfg.h | Generated file that contains type definitions and extern declarations of the initialization tables. | - |
| vLinkGen_Cfg.h | Generated file that contains pre-compile time macros. | - |

Table 3-1 Implementation files

In addition to the source code files mentioned above, vLinkGen also generates one or more compiler specific linker scripts or snippets.

The following Table 3-2 shows the different files that can be generated. Which of these variants is generated depends on the **File Generation** parameter, which is described in chapter 2.1.

| File Name | Description | Integration Tasks |
|---|---|---|
| vLinkGen_Template.<ext> | Compiler specific linker script. | May have to be preprocessed and then passed to linker via specific option. |
| vLinkGen_Template_<Variant>.<ext> | Compiler specific linker script for a dedicated variant. Where <Variant> is the name of the variant. | Must be passed to linker via specific option. |
| vLinkGen_Inc_<Name>.<ext> | Compiler specific linker script snippet. Where <Name> can be the name of a module, variant or Logical Group. | Must be included within another linker script. Cannot be used stand-alone. |

Table 3-2 Generated linker scripts and snippets

Where <ext> is the compiler specific file extension for linker scripts.



**Note**
The linker scripts are generated in the **“Template Files”** (Source) folder of the DaVinci Configurator Project.

These files are to be seen as templates and can be changed manually at any time.

Please note that the files will be overwritten any time the vLinkGen configuration changes. Therefore, if the files are to be modified manually, they should be copied to another location.

## 3.2 Compiler Specialities

The following chapter indicates the different compiler specialties. For example, which features are not supported at all or only partially. Or which special flags can be set for the respective compilers.

Parameters which are set to read-only in the DaVinci Configurator are not supported for the selected compiler.

### 3.2.1 ARM

- End Alignments (`vLinkGen*SectionGroupEndAlignment`) are not generated.
- The vLinkGen does not generate any labels (e.g. for Section Groups) for this compiler. However, the linker itself creates start and end labels for each Section Group. For details, please refer to chapter 3.4.
- The compiler/linker must be called with option `--bss_threshold=0`. Otherwise non-initialized variables might be handled as initialized data (`rwdata`). This is only confirmed for ARM v5.
- In the ARM scatter file the Logical Groups within a Memory Region Block will be always sorted by their
  - 1) type (in the order CONST, VAR, USER, SYMBOL)
  - 2) position
  - 3) name

#### 3.2.1.1 Reordered Groups

The actual order of the Logical Groups in the linker script may differ from the configured position (`vLinkGenLogical*GroupPosition`). This is a limitation of the ARM scatter file syntax. More details can be found in the following example:



**Example**
The following example shows why it is not possible to always guarantee the order in ARM scatter files:

```
CodeFlash_B 0x20000000 0x00400000
{
    GlobalRam_A_AD 0xFEEF8000 EMPTY 0 { }
    pos_one_data +0 { … } //LogicalGroup, position 1
    pos_two_data +0 { … } //LogicalGroup, position 2
    GlobalRam_A_AD_END +0 EMPTY 0 { }
}

CodeFlash_A 0x10000000 0x00400000
{
    GlobalRam_A_1_AD ImageLimit(GlobalRam_A_AD_END)EMPTY 0{ }
    pos_zero_data +0 { … } //LogicalGroup, position 0
    pos_five_data +0 { … } //LogicalGroup, position 5
    GlobalRam_A_1_AD_END +0 EMPTY 0 { }
}
```

In this example all Logical Groups shall be executed in the Memory Region Block `GlobalRam_A` (with address `0xFEEF8000`) and are initialized from different Initialization Memory Region Blocks (`CodeFlash_B` and `CodeFlash_A`). When generated as above, the positions are violated because the Logical Group with position 0 is behind the groups with position 1 and 2. If the generation order of `CodeFlash_B` and `CodeFlash_A` is exchanged, the problem still exists, because the Logical Groups with position 1 and 2 are behind the group with position 5.

The order of the Section Groups within a Logical Group, however, is guaranteed.

If the order of certain or all Logical Groups within a Memory Region Block must be stictly adhered to, the two options described in 3.2.4.2.1 and 3.2.4.2.2 can be used.

### 3.2.2 Green Hills

- The "DEFAULTS" area in the linker script is currently not supported.

### 3.2.3 Renesas

- Alignments are only generated for Linker Sections and are restricted to 16 bytes.
- The vLinkGen does not generate any labels (e.g. for Section Groups) for this compiler. However, the linker itself creates start and end labels for each Linker Section. For details, please refer to chapter 3.4.

### 3.2.4 Tasking

- The linker must be called with option `-Wl--user-provided-initialization-code`.
- If the linker is called with option `--section-name-with-symbol`, all configured Linker Sections might need a wildcard (*) configured in their **Filenames** (`vLinkGenLinker*SectionFiles`) parameter. Otherwise the Linker Sections might be linked to another location then expected.
- The functionality of the **KEEP** flag is only guaranteed from version v6.1r1 of the Tasking Compiler. It may work with earlier versions also, but this is not guaranteed. The KEEP flag can be configured for any vLinkGenLinkerSections via the **Compiler Specific Flags** (`vLinkGenLinker*SectionFlag`) parameter.

#### 3.2.4.1 Special Flags

In addition to the compiler specific flags (e.g., `ORDERED`, `CONTIGUOUS`, etc.), vLinkGen provides some additional special flags to be configured. The following table describes the special flags available for Tasking with their meaning and where they can be configured.

For details about the compiler specific flags, please refer to the corresponding compiler manual.

| Flag | Configuration Location | Description |
|---|---|---|
| `FORCE_ZERO_OFFSET` | Logical Const/Var Group | When set, the zero offset (`[0]`) is generated for this logical group at the `run_addr` attribute. Only if it is the first logical group in the corresponding memory region block.<br>This forces the linker to link the `group` at the start address of the memory region block and not reorder it (see also 3.2.4.2). |

Table 3-3 Tasking Special Flags

#### 3.2.4.2 Reordered Groups after Linking

The actual order of the Logical Groups after linking may differ from the configured position (`vLinkGenLogical*GroupPosition`). This is a restriction of the linker and the way vLinkGen generates its linker scripts. The linker optimizes the order of the Logical Groups depending on their sizes and alignments to keep the memory loss as low as possible.

The order of the Section Groups within a Logical Group, however, is guaranteed.

Likewise, it is ensured that the first Logical Group of a Memory Region Block is always linked at the beginning. If a single Logical Group shall be linked at the beginning of a Memory Region Block, it can simply be assigned the position 0. For more details about the output generation order please see 3.3.

If the order of certain or all Logical Groups within a Memory Region Block must be strictly adhered to, there are two ways to ensure this:

#### 3.2.4.2.1 Combine Logical Groups

The easiest way to ensure the order of the Logical Groups within a Memory Region Block is to merge all Section Groups from these Logical Groups into one or more Logical Groups.

This can be done in the DaVinci Configurator via copy and paste of the existing Section Groups (`vLinkGen*SectionGroup`) into the corresponding Logical Group(s). After that, the positions of the Section Groups can be configured as desired, and all superfluous Logical Groups can be removed.

#### 3.2.4.2.2 Split Memory Regions

The other solution is to divide Memory Regions into several Memory Region Blocks and to link the Logical Groups to them accordingly. This is particularly necessary if Logical Groups shall be linked with absolute addresses.

Disadvantage, however, is that the sizes (`vLinkGenMemoryRegionBlockSize`) of the Memory Region Blocks must be predicted during configuration time. What means, the sizes may need to be adjusted after the first linking. Or even if the content of a linker section changes later in the process, adjusting the Memory Region Block size might be necessary.

### 3.2.4.3 Workaround for TCVX-39968

The following chapter describes a workaround for the following Tasking linker issue:

"*TCVX-39968 When using GROUPing, the order of the ROM copies of initialized data is different to their RAM order*".

#### 3.2.4.3.1 Error Description

The order of the variables of a RAM group differs in the created ROM copy. This can cause the variables to be initialized with incorrect values, which in turn can lead to unexpected behavior. The re-order is caused when using the wildcard operator (e.g. "`[.]data*`") and if variables are present where one name contains the other completely and from the beginning, for example:

> `.data.file.GlobalVar`

> `.data.file.GlobalVar`**`Ext`**

The following extract of a map file shall illustrate the issue. Where the order of two global variables in RAM (`Data_Default`) and ROM (`Data_Default_INIT_ROM`) is reversed. Where sections in brackets ("`[...]`") are always the ROM copies of initialized RAM sections created by the linker.



**Extract of Tasking map file**

```
...
********************************  Locate Result  ********************************
...
| mpe:dspr0 | Data_Default          | .data.file.GlobalVar      | 0x7000af20 | ... |
| mpe:dspr0 | Data_Default          | .data.file.GlobalVarExt   | 0x7000af2C | ... |
...
| mpe:flash | Data_Default_INIT_ROM | [.data.file.GlobalVarExt] | 0xa0066480 | ... |
| mpe:flash | Data_Default_INIT_ROM | [.data.file.GlobalVar]    | 0xa006648C | ... |
...
```

#### 3.2.4.3.2 Preconditions

The issue only occurs if the following conditions are met:

- The used Tasking compiler version is v4.2.r2 or lower.
- The source code contains global variables which are not linked to a user defined linker section (e.g. via AUTOSAR MemMap mechanism)
  - Those variables will then automatically be linked into the `Data_Default` section group, respectively at the position of "`select "[.]data*"`".

#### 3.2.4.3.3 Workaround

To work around this issue, all affected variables must be specified explicitly in the linker script.

1. Search for variables in the project map file that meet the above-mentioned preconditions and note them.
2. Create a new `vLinkGenLinkerSpecSection` for each variable with the **Short-Name** set to the name of the variable (e.g. `GlobalVar`).
   1. Set the **Value** (`vLinkGenLinkerSpecSectionValue`) of the section to the complete section name from the map file (e.g. `.data.file.GlobalVar`). Where `file` is the name of the object file in which the variable is defined.
3. Duplicate the `vLinkGenVarSectionGroup` **data** and remove all referenced **Sections** from the new one. The Section Group is a child of the `vLinkGenLogicalVarGroup` **Data_Default**. The name of the new Section Group could be **data_GlobalVars**.
   1. Ensure that **Init Policy** and **Init Stage** parameters have the same values as in the **data** Section Group.
4. Then add all created `vLinkGenLinkerSpecSections` as **Sections** to the new **data_GlobalVars** Section Group.
5. Finally, set the **Position** of the new **data_GlobalVars** Section Group to a value lower than the position of the **data** Section Group. This ensures that it is generated and

linked before the **data** Section Group. Otherwise, the wildcard operator within **data** will also match the explicitly specified linker sections.

### 3.2.5 Texas Instruments

- The actual order of the Section Groups after linking may differ from the configured position (`vLinkGen*GroupPosition`). This is a restriction of the compiler and cannot be influenced by vLinkGen.

## 3.3 Output Generation Order

The following table provides an overview of the individual output types and their order in the generated linker script.

| Output Type | Config Container | Sort Order |
|---|---|---|
| Memory Region Blocks | `vLinkGenMemoryRegionBlock` | All Memory Region Blocks within a Memory Region are sorted by their boundary (lower, upper) and then by position. |
| Logical Groups | `vLinkGenLogicalConstGroup`<br>`vLinkGenLogicalSymbolGroup`<br>`vLinkGenLogicalUserGroup`<br>`vLinkGenLogicalVarGroup` | All Logical Groups are sorted by their position and then by container name. Whereby the list of all Logical Groups, regardless of type, is sorted. |
| Section Groups | `vLinkGenConstSectionGroup`<br>`vLinkGenUserSectionGroup`<br>`vLinkGenVarSectionGroup` | All Section Groups within a Logical Group are sorted by their position and then by container name. |
| Linker Sections | `vLinkGenLinkerCodeSection`<br>`vLinkGenLinkerConstSection`<br>`vLinkGenLinkerSpecSection`<br>`vLinkGenLinkerStackSection`<br>`vLinkGenLinkerVarSection` | All Linker Sections within a Section Group are sorted by their container name. |
| Linker Symbols | `vLinkGenLinkerSymbol` | All Linker Symbols within a Logical Symbol Group are sorted by their configured name. |

Table 3-4 Output generation order



**Note**
The order of the Section Groups in the generated linker script also depends on the position of their parent Logical Group. These are also sorted internally by their position and then by their name.

## 3.4 Label Generation

The following table shows an overview of the generated labels per compiler and which values they get after linking. The shown labels are either created by the respective compiler or added by vLinkGen.

The table is not complete. There may be other compiler specific labels. Please refer to the corresponding compiler manual for details.

| Compiler | Label | Value |
|---|---|---|
| ARM | Image$$**<Name>**$$Base | Start address |
| | Image$$**<Name>**$$Limit | Start address + Size |
| Ccac, Diab, Green Hills, GCC, Linaro, HighTec, Tasking, Texas Instruments | [_]_**<Name>**_START | Start address |
| | [_]_**<Name>**_END | Start address + Size - 1 |
| | [_]_**<Name>**_LIMIT | Start address + Size |
| IAR | [_]_**<Name>**_START (**<Name>**$$Base) | Start address |
| | [_]_**<Name>**_LIMIT (**<Name>**$$Limit) | Start address + Size |
| Renesas | __S_**<Name>** | Start address |
| | __E_**<Name>** | Start address + Size |

Table 3-5 Label Generation

**Note**

Where **<Name>** can be the name of a Memory Region Block, Section Group or Linker Section.

The LIMIT labels are used for the initialization tables.

The END labels are usually used for configuring end addresses of MPU regions. Because MPU region end addresses mostly need to be set up with the last writable address.

In addition, the vLinkGen generates also labels for all Logical Groups like the following:

```
[_]_<LogicalGroupName>_ALL_[START|END|LIMIT]
```

Those labels wrap all Section Groups configured within one Logical Group. That means, the START label points to the _<SectionGroupName>_START label of the first Section Group. The END and LIMIT labels point to the respective labels of the last Section Group.



**Caution**
These Logical Group labels are to be used with care (e.g. for MPU configuration) as it is not guaranteed, that the linker links all Section Groups, respectively all Linker Sections of a Logical Group next to each other.

## 3.5 Special Section Generation

The following table shows how Linker Sections with special values (`vLinkGenLinkerSpecSection`) are generated in the compiler specific script language. Where parts in brackets ("`[…]`") are optional and only generated if configured. Strings in angle brackets ("<…>") are replaced with the corresponding parameter values.

An example how to configure a section with special values can be found in chapter 5.

| Compiler | Linker Script Syntax |
|---|---|
| ARM | `*(<Value>[, OVERALIGN <Alignment>])` |
| Ccac, Diab | `.<Short-Name>[ KEEP][ (NOLOAD)][ ALIGN(<Alignment>)] : { <Value> }` |
| GCC | `[. = ALIGN(<Alignment>);] [KEEP()*(<Value>)[])]` |
| Green Hills, HighTec, Texas Instruments | `[. = ALIGN(<Alignment>);] *(<Value>)` |
| IAR | `Block <Short-Name>_BLOCK { section <Value> },` |
| Renesas | `<Value>[, &]` |
| Tasking | `select "<Value>";` |

Table 3-6 Special Section Generation

## 3.6 Migration of Existing Projects

For existing DaVinci Configurator projects containing a configuration of the vLinkGen in version 1.07.x, the version 2.xx.xx provides the possibility to migrate this configuration to the new BSWMD structure. This means that the first time this project is opened with the new vLinkGen version, the configuration will be automatically adjusted to fit the changed BSWMD and only minimal manual adjustments are necessary.



**Note**
The preferred way to migrate an existing project would be to add vLinkGen completely new to the project. This means first removing the module from the project settings and then adding it again.

However, this way should only be used if no manual changes have been made to the vLinkGen configuration (e.g. adding custom linker sections). Otherwise they will be deleted with the module. In this case, the migration should be performed using the following steps.

The following conditions must be observed before migration:

- The version of vLinkGen must have been at least 1.07.xx before.
- The vBaseEnv must be upgraded to version 2.xx.xx.
- If the MICROSAR OS is included in the project, it must also be upgraded to version 2.45.00 or higher.
- The existing configuration of vLinkGen must not have any errors.
  With warnings it may work, but this is not guaranteed.

To perform the migration, the following steps must be performed:

1. The vLinkGen, vBaseEnv and possibly OS files in the SIP must be exchanged with the new files before the migration.
2. The `<SipId>` in the project file of the DaVinci Configurator must be changed. This should be done automatically for update deliveries.
3. Open the project with the DaVinci Configurator.
4. If the changes are recognized correctly, a pop-up window named **SIP Mismatch** will appear.
5. Select the option **Migrate the project to this SIP and continue** and start migration with **OK**. This may take some time.
6. After the migration some error messages may appear in the **Validation** window. These can be easily fixed with the **Solve All** button. This may take some time again, depending on the size of the configuration.
7. Now remaining error messages of the vLinkGen must be fixed manually, because it can't find a suitable solution at this point. The displayed error messages should be the following:
   - **Cfg00022 - Missing parameter value**

     The Memory Region Block reference must be selected for each Logical Group.
   - **VLINKGEN01300 - Invalid init core ID configuration**

     These errors can all be solved simultaneously via the Solve all with ... option.
   - **VLINKGEN03260 - Invalid VAR section group init configuration**

The Memory Region Block (in ROM) in which the initialization data of variables should be located must be selected here.

8. The configuration is now successfully migrated, and the new linker script can be generated.



**Note**
Only the configuration of the vLinkGen module is changed during the migration. All other modules are not affected by the changes.

# 4 API Description

## 4.1 Type Definitions

The types defined by the vLinkGen are described in this chapter.

### 4.1.1 vLinkGen_MemArea

This structure contains the information for a generic memory area to be initialized.

| Struct Element Name | C-Type | Description | Value Range |
|---|---|---|---|
| Start | `uint32` | Start address of the memory area. | 0..4294967295 |
| End | `uint32` | End address of the memory area. Calculated from start address + size of the area. | 0..4294967295 |
| Core | `uint32` | The physical core ID to perform the initialization. | 0..4294967295 |
| Alignment | `uint32` | Optional ECC alignment of the Hardware Memory Area in bytes. If no alignment is set, the value is generated to zero. | 0..4294967295 |

Table 4-1 vLinkGen_MemArea

### 4.1.2 vLinkGen_MemAreaSet

This structure contains the information for a set of generic memory areas.

| Struct Element Name | C-Type | Description | Value Range |
|---|---|---|---|
| Num | `uint32` | Number of areas. | 0..4294967295 |
| Areas | `vLinkGen_MemArea` | Table of generic memory areas. See 4.1.1 type definition. | - |

Table 4-2 vLinkGen_MemAreaSet

### 4.1.3 vLinkGen_RamMemArea

This structure contains the information for a RAM memory area to be initialized with concrete data from a ROM area.

| Struct Element Name | C-Type | Description | Value Range |
|---|---|---|---|
| Start | `uint32` | Start address of the RAM memory area. | 0..4294967295 |
| End | `uint32` | End address of the RAM memory area. Calculated from start address + size of the RAM area. | 0..4294967295 |
| Romstart | `uint32` | Start address of the ROM memory area. | 0..4294967295 |
| Romend | `uint32` | End address of the ROM memory area. Calculated from start address + size of the ROM area. | 0..4294967295 |
| Core | `uint32` | The physical core ID to perform the initialization. | 0..4294967295 |

| Struct Element Name | C-Type | Description | Value Range |
|---|---|---|---|
| Alignment | `uint32` | Optional ECC alignment of the Hardware Memory Area in bytes. If no alignment is set, the value is generated to zero. | 0..4294967295 |

Table 4-3 vLinkGen_RamMemArea

### 4.1.4 vLinkGen_RamMemAreaSet

This structure contains the information for a set of ROM to RAM copy memory areas.

| Struct Element Name | C-Type | Description | Value Range |
|---|---|---|---|
| Num | `uint32` | Number of areas. | 0..4294967295 |
| Areas | `vLinkGen_RamMemArea` | Table of ROM to RAM copy memory areas.<br>See 4.1.3 for type definition. | - |

Table 4-4 vLinkGen_RamMemAreaSet

# 5 Configuration

This chapter was moved to the vBaseEnv User Manual [2] (as chapter “vLinkGen”).

# 6 Glossary and Abbreviations

## 6.1 Glossary

| Term | Description |
|---|---|
| BSWMD | The BSWMD is a formal notation of all information belonging to a certain BSW artifact (BSW module or BSW cluster) in addition to the implementation of that artifact. |
| Electronic Control Unit | Also known as ECU. Small embedded computer system consisting of at least one CPU and corresponding periphery which is placed in one housing. |
| Linker Script | Compiler specific description file which is used by the linker to know where to allocate sections in the target's memory. In compiler manuals often called “Linker Command File” or “Linker Description”. |
| Operating system | All management and executive programs of a computer and its resources comprising an available and clearly defined interface. |

Table 6-1 Glossary

## 6.2 Abbreviations

| Abbreviation | Description |
|---|---|
| API | Application Programming Interface |
| AUTOSAR | Automotive Open System Architecture |
| BSW | Basis Software |
| CAN | Controller Area Network protocol originally defined for use as a communication network for control applications in vehicles. |
| ECC | Error-Correcting Code memory |
| ECU | Electronic Control Unit |
| HIS | Hersteller Initiative Software |
| ID | Identifier (e.g. Identifier of a CAN message) |
| MICROSAR | Microcontroller Open System Architecture (the Vector AUTOSAR solution) |
| OS | Operating System |
| PLL | Phase-locked loop |
| RAM | Random Access Memory |
| ROM | Read Only Memory |
| SIP | Software Integration Package |
| vBRS | Vector Basic Runtime System |

Table 6-2 Abbreviations

# 7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com